Índice

Listado de objetos utilitarios
Objeto #dates
Objeto #numbers
Objeto #strings
Objeto #lists
Objeto #maps
Objeto #bools
Otros objetos utilitarios

Listado de objetos utilitarios

Objeto utilitario	Tipo de datos que maneja	Propósito principal	Ejemplo de uso	Resultado esperado
`#strings`	Cadenas de texto (`String`)	Manipulación y formato de texto: mayúsculas, búsqueda, reemplazo, concatenación, etc.	`[[${#strings.toUpperCase('hola')}]]`	`HOLA`
`#numbers`	Números (`int`, `double`, etc.)	Formato y operaciones con números: decimales, monedas, porcentajes, redondeos, etc.	`[[${#numbers.formatDecimal(12.3,1,2)}]]`	`12.30`
`#dates`	Fechas (`java.util.Date`)	Formatear y obtener partes de una fecha (día, mes, año, etc.).	`[[${#dates.format(hoy, 'dd/MM/yyyy')}]]`	`05/10/2025`
`#dates.calendar`	Calendarios (`java.util.Calendar`)	Obtener y manipular información del calendario (semana, día, hora…).	`[[${#dates.weekOfYear(#dates.calendar())}]]`	`40`
`#calendars`	Calendarios (`Calendaro GregorianCalendar`)	Comparar fechas, calcular diferencias, sumar o restar periodos.	`[[${#calendars.diffDays(cal1, cal2)}]]`	`10`
`#temporals`	Fechas modernas (`LocalDate`,	Manipular y formatear fechas de la API Java 8+ (`java.time`).	`[[${#temporals.format(#temporals.createNow(), 'dd/MM/yyyy')}]]`	`05/10/2025`

	`LocalDateTime`, `Instant`)			
`#lists`	Listas (`List`)	Operaciones sobre listas: tamaño, orden, búsqueda, concatenación, sublistas, etc.	`[[${#lists.sort(nombres)}]]`	`[Ana, Luis, Marta]`
`#arrays`	Arreglos (`String[]`, `int[]`, etc.)	Trabajar con arrays: tamaño, acceso por índice, orden, unión, conversión a lista.	`[[${#arrays.get(numeros,1)}]]`	`20`
`#maps`	Mapas o diccionarios (`Map<K,V>`)	Acceso a pares clave–valor, unión de mapas, comprobaciones, iteraciones.	`[[${#maps.get(datos, 'nombre')}]]`	`Carlos`
`#bools`	Booleanos (`true/false`)	Evaluar condiciones lógicas: `and`, `or`, `not`, `isTruthy`, `isFalsy`.	`[[${#bools.and(true, false)}]]`	`false`
`#aggregates`	Colecciones numéricas (listas o arrays)	Calcular agregados: suma, promedio, mínimo, máximo, producto, moda, mediana.	`[[${#aggregates.sum(List.of(2,4,6))}]]`	`12`
`#dates.create`, `#dates.addDays`, etc.	Fechas (`Date`)	Crear y modificar fechas específicas.	`[[${#dates.addDays(hoy, 7)}]]`	`12/10/2025`
`#ids`	Identificadores únicos	Generar IDs únicos para etiquetas HTML.	`th:id="${#ids.seq('item')}"`	`item1`, `item2`, ...

#messages	Archivos de propiedades (`messages.properties`)	Obtener textos internacionalizados (i18n).	`[[#{saludo.mensaje}]]`	`¡Hola!`
#uris	URLs	Construcción y manipulación de rutas y parámetros en URLs.	`[[${#uris.buildURI('/producto', {'id':5})}]]`	`/producto?id=5`
#execInfo	Información de ejecución	Información sobre la plantilla y el contexto actual.	`[[${#execInfo.templateName}]]`	`index.html`
#ctx	Contexto de ejecución (`IContext`)	Acceso a variables del contexto actual.	`[[${#ctx.variables}]]`	`{nombre=Carlos, edad=25}`
#locale	Configuración regional	Devuelve el locale actual (idioma y país).	`[[${#locale}]]`	`es_ES`
#objects	Acceso a objetos del entorno	Permite acceder a beans u objetos registrados.	`[[${#objects['beanNombre'].propiedad}]]`	valor
#environment	Variables del entorno	Acceso a propiedades del entorno de Spring (`application.properties`).	`[[${#environment.getProperty('spring.application.name')}]]`	`MiAplicacion`

Objeto #dates

Por categorías:

Categoría	Funciones más útiles
Creación	`create()`, `createNow()`, `createToday()`
Formato	`format()`, `formatISO()`
Partes de la fecha	`day()`, `month()`, `year()`, `hour()`, `minute()`
Nombres	`monthName()`, `dayOfWeekName()`
Operaciones	`addDays()`, `addMonths()`, `addYears()`
Comparaciones	`isBefore()`, `isAfter()`, `isSameDay()`

Detalladas:

Método / Propiedad	Descripción	Ejemplo de uso	Resultado esperado
`#dates.create(year, month, day)`	Crea una nueva fecha con año, mes y día.	`[[${#dates.create(2025, 10, 5)}]]`	`Sun Oct 05 00:00:00 CEST 2025`

#dates.format(date)	Devuelve la fecha en formato por defecto (`yyyy-MM-dd HH:mm:ss`).	`[[${#dates.format(hoy)}]]`	`2025-10-05 14:22:00`
#dates.format(date, pattern)	Devuelve la fecha en el formato indicado.	`[[${#dates.format(hoy, 'dd/MM/yyyy')}]]`	`05/10/2025`
#dates.formatISO(date)	Formatea la fecha en formato ISO-8601.	`[[${#dates.formatISO(hoy)}]]`	`2025-10-05T14:22:00`
#dates.day(date)	Devuelve el día del mes (1–31).	`[[${#dates.day(hoy)}]]`	`5`
#dates.month(date)	Devuelve el mes (1–12).	`[[${#dates.month(hoy)}]]`	`10`
#dates.monthName(date)	Devuelve el nombre del mes (en inglés por defecto).	`[[${#dates.monthName(hoy)}]]`	`October`
#dates.year(date)	Devuelve el año de la fecha.	`[[${#dates.year(hoy)}]]`	`2025`
#dates.dayOfWeek(date)	Devuelve el día de la semana (1=Lunes, 7=Domingo).	`[[${#dates.dayOfWeek(hoy)}]]`	`7`
#dates.dayOfWeekName(date)	Devuelve el nombre del día de la semana.	`[[${#dates.dayOfWeekName(hoy)}]]`	`Sunday`
#dates.time(date)	Devuelve la hora del día (HH:mm:ss).	`[[${#dates.time(hoy)}]]`	`14:22:00`
#dates.hour(date)	Devuelve la hora (0–23).	`[[${#dates.hour(hoy)}]]`	`14`

#dates.minute(date)	Devuelve los minutos (0–59).	`[[${#dates.minute(hoy)}]]`	`22`
#dates.second(date)	Devuelve los segundos (0–59).	`[[${#dates.second(hoy)}]]`	`30`
#dates.createNow()	Crea la fecha y hora actual.	`[[${#dates.createNow()}]]`	`Sun Oct 05 14:22:00 CEST 2025`
#dates.createToday()	Crea una fecha con el día actual (hora = 00:00:00).	`[[${#dates.createToday()}]]`	`Sun Oct 05 00:00:00 CEST 2025`
#dates.addDays(date, n)	Suma (o resta) días a una fecha.	`[[${#dates.format(#dates.addDays(hoy, 7), 'dd/MM/yyyy')}]]`	`12/10/2025`
#dates.addMonths(date, n)	Suma o resta meses.	`[[${#dates.format(#dates.addMonths(hoy, -1), 'dd/MM/yyyy')}]]`	`05/09/2025`
#dates.addYears(date, n)	Suma o resta años.	`[[${#dates.format(#dates.addYears(hoy, 1), 'dd/MM/yyyy')}]]`	`05/10/2026`
#dates.isBefore(date1, date2)	Comprueba si una fecha es anterior a otra.	`[[${#dates.isBefore(fecha1, fecha2)}]]`	`true` o `false`
#dates.isAfter(date1, date2)	Comprueba si una fecha es posterior a otra.	`[[${#dates.isAfter(fecha1, fecha2)}]]`	`true` o `false`
#dates.isSameDay(date1, date2)	Comprueba si dos fechas son el mismo día.	`[[${#dates.isSameDay(fecha1, fecha2)}]]`	`true` o `false`

Objeto #numbers

Por categorías:

Categoría	Métodos más útiles
Formato de números	`formatInteger()`, `formatDecimal()`, `formatCurrency()`, `formatPercent()`, `formatScientific()`
Cálculos básicos	`round()`, `floor()`, `ceiling()`, `abs()`, `min()`, `max()`
Listas numéricas	`sum()`, `avg()`, `sequence()`
Condiciones	`isEven()`, `isOdd()`
Formatos personalizados	`formatNumber(numero, patrón)` → Ej: `#,###.00`

Detalladas:

Método / Propiedad	Descripción	Ejemplo de uso	Resultado
`#numbers.formatInteger(numero)`	Muestra un número entero sin decimales.	`[[${#numbers.formatInteger(1234)}]]`	`1,234`
`#numbers.formatDecimal(numero, minInt, minFrac)`	Formatea un número decimal con número mínimo de enteros y decimales.	`[[${#numbers.formatDecimal(12.3, 3, 2)}]]`	`012.30`
`#numbers.formatDecimal(numero, minInt, minFrac, maxFrac)`	Controla número mínimo/máximo de decimales.	`[[${#numbers.formatDecimal(12.3456, 1, 2, 3)}]]`	`12.346`
`#numbers.formatCurrency(numero)`	Formatea como moneda según la configuración regional.	`[[${#numbers.formatCurrency(1234.5)}]]`	`$1,234.50` *(en USA)*
`#numbers.formatPercent(numero)`	Formatea el número como porcentaje.	`[[${#numbers.formatPercent(0.25)}]]`	`25%`
`#numbers.formatScientific(numero)`	Muestra el número en notación científica.	`[[${#numbers.formatScientific(12345)}]]`	`1.2345E4`
`#numbers.formatNumber(numero, pattern)`	Permite aplicar un patrón personalizado.	`[[${#numbers.formatNumber(1234.56, '#,###.00')}]]`	`1,234.56`
`#numbers.sequence(from, to)`	Genera una secuencia de enteros consecutivos.	`[[${#numbers.sequence(1, 5)}]]`	`[1, 2, 3, 4, 5]`
`#numbers.sequence(from, to, step)`	Genera una secuencia con paso definido.	`[[${#numbers.sequence(0, 10, 2)}]]`	`[0, 2, 4, 6, 8, 10]`

#numbers.isEven(numero)	Devuelve `true` si el número es par.	`[[${#numbers.isEven(4)}]]`	`true`
#numbers.isOdd(numero)	Devuelve `true` si el número es impar.	`[[${#numbers.isOdd(7)}]]`	`true`
#numbers.abs(numero)	Devuelve el valor absoluto.	`[[${#numbers.abs(-10)}]]`	`10`
#numbers.round(numero)	Redondea al entero más cercano.	`[[${#numbers.round(3.6)}]]`	`4`
#numbers.floor(numero)	Redondea hacia abajo (entero menor o igual).	`[[${#numbers.floor(3.9)}]]`	`3`
#numbers.ceiling(numero)	Redondea hacia arriba (entero mayor o igual).	`[[${#numbers.ceiling(3.1)}]]`	`4`
#numbers.min(num1, num2)	Devuelve el menor de dos números.	`[[${#numbers.min(3, 8)}]]`	`3`
#numbers.max(num1, num2)	Devuelve el mayor de dos números.	`[[${#numbers.max(3, 8)}]]`	`8`
#numbers.avg(lista)	Calcula el promedio de una lista numérica.	`[[${#numbers.avg(List.of(2,4,6))}]]`	`4.0`
#numbers.sum(lista)	Suma los valores de una lista.	`[[${#numbers.sum(List.of(2,4,6))}]]`	`12.0`

Objeto #strings

Por categorías:

Categoría	Métodos más útiles
Transformación	`toUpperCase()`, `toLowerCase()`, `capitalize()`, `unCapitalize()`
Información	`length()`, `isEmpty()`, `isEmptyOrWhitespace()`
Comparación / búsqueda	`equals()`, `equalsIgnoreCase()`, `contains()`, `startsWith()`, `endsWith()`, `indexOf()`
Manipulación	`substring()`, `trim()`, `replace()`, `concat()`, `reverse()`
Formatos especiales	`abbreviate()`, `defaultString()`
Listas y arrays	`arrayJoin()`, `listJoin()`
Aleatorios	`randomAlphanumeric()`

Detallada:

Método / Propiedad	Descripción	Ejemplo de uso	Resultado esperado
`#strings.toUpperCase(texto)`	Convierte el texto a mayúsculas.	`[[${#strings.toUpperCase('hola')}]]`	`HOLA`
`#strings.toLowerCase(texto)`	Convierte el texto a minúsculas.	`[[${#strings.toLowerCase('HOLA')}]]`	`hola`
`#strings.capitalize(texto)`	Pone en mayúscula la primera letra.	`[[${#strings.capitalize('primavera')}]]`	`Primavera`
`#strings.unCapitalize(texto)`	Pone en minúscula la primera letra.	`[[${#strings.unCapitalize('Invierno')}]]`	`invierno`
`#strings.length(texto)`	Devuelve la longitud de la cadena.	`[[${#strings.length('Hola')}]]`	`4`
`#strings.isEmpty(texto)`	Devuelve `true` si la cadena está vacía.	`[[${#strings.isEmpty('')}]]`	`true`
`#strings.isEmptyOrWhitespace(texto)`	Devuelve `true` si está vacía o solo tiene espacios.	`[[${#strings.isEmptyOrWhitespace(' ')}]]`	`true`
`#strings.equals(texto1, texto2)`	Compara dos cadenas.	`[[${#strings.equals('uno','uno')}]]`	`true`
`#strings.equalsIgnoreCase(t1, t2)`	Compara ignorando mayúsculas/minúsculas.	`[[${#strings.equalsIgnoreCase('UNO','uno')}]]`	`true`

`#strings.contains(texto, subtexto)`	Comprueba si una cadena contiene otra.	`[[${#strings.contains('Primavera', 'ma')}]]`	`true`
`#strings.startsWith(texto, prefijo)`	Comprueba si empieza con una subcadena.	`[[${#strings.startsWith('Hola mundo','Hola')}]]`	`true`
`#strings.endsWith(texto, sufijo)`	Comprueba si termina con una subcadena.	`[[${#strings.endsWith('archivo.txt','.txt')}]]`	`true`
`#strings.indexOf(texto, subtexto)`	Devuelve la posición (índice) de la subcadena.	`[[${#strings.indexOf('manzana','za')}]]`	`4`
`#strings.substring(texto, inicio, fin)`	Devuelve una parte de la cadena.	`[[${#strings.substring('manzana',0,3)}]]`	`man`
`#strings.trim(texto)`	Elimina espacios en blanco al inicio y final.	`[[${#strings.trim(' hola ')}]]`	`hola`
`#strings.concat(t1, t2)`	Une dos cadenas.	`[[${#strings.concat('Hola ', 'Mundo')}]]`	`Hola Mundo`
`#strings.replace(texto, viejo, nuevo)`	Reemplaza un texto por otro.	`[[${#strings.replace('rojo', 'ro', 've')}]]`	`vejo`
`#strings.reverse(texto)`	Invierte el orden de los caracteres.	`[[${#strings.reverse('sol')}]]`	`los`
`#strings.abbreviate(texto, tamaño)`	Corta la cadena y añade “…” si es más larga que el tamaño.	`[[${#strings.abbreviate('Primavera', 5)}]]`	`Pr...`

`#strings.randomAlphanumeric(n)`	Genera una cadena aleatoria con letras y números.	`[[${#strings.randomAlphanumeric(8)}]]`	`A7b92C1x`
`#strings.arrayJoin(array, separador)`	Une los elementos de un array o lista con un separador.	`[[${#strings.arrayJoin(nombres, ', ')}]]`	`Ana, Luis, Marta`
`#strings.listJoin(lista, separador)`	Igual que `arrayJoin`, pero para listas.	`[[${#strings.listJoin(List.of('A','B','C'), '-')}]]`	`A-B-C`
`#strings.defaultString(texto, valorDefecto)`	Devuelve un valor por defecto si la cadena es `null`.	`[[${#strings.defaultString(nombre, 'Desconocido')}]]`	Desconocido

Objeto #lists

Por categorías:

Categoría	Métodos más útiles
Información básica	`size()`, `isEmpty()`, `contains()`
Acceso directo	`first()`, `last()`, `get()`
Orden y selección	`sort()`, `reverse()`, `sublist()`
Operaciones entre listas	`concat()`, `union()`, `intersect()`, `diff()`
Cálculos numéricos	`sum()`, `avg()`, `min()`, `max()`
Otros	`shuffle()` (aleatorio), `containsAll()`

Detallada:

Método / Propiedad	Descripción	Ejemplo de uso	Resultado esperado
`#lists.size(lista)`	Devuelve el tamaño (número de elementos) de la lista.	`[[${#lists.size(nombres)}]]`	`3`
`#lists.isEmpty(lista)`	Devuelve `true` si la lista está vacía.	`[[${#lists.isEmpty(nombres)}]]`	`false`
`#lists.contains(lista, elemento)`	Comprueba si la lista contiene un elemento.	`[[${#lists.contains(nombres, 'Ana')}]]`	`true`
`#lists.containsAll(lista1, lista2)`	Devuelve `true` si todos los elementos de `lista2` están en `lista1`.	`[[${#lists.containsAll(lista1, lista2)}]]`	`true`
`#lists.sort(lista)`	Devuelve una nueva lista ordenada (alfabéticamente o numéricamente).	`[[${#lists.sort(nombres)}]]`	`[Ana, Luis, Marta]`
`#lists.reverse(lista)`	Devuelve una nueva lista en orden inverso.	`[[${#lists.reverse(nombres)}]]`	`[Marta, Luis, Ana]`
`#lists.first(lista)`	Devuelve el **primer elemento** de la lista.	`[[${#lists.first(nombres)}]]`	`Ana`
`#lists.last(lista)`	Devuelve el **último elemento** de la lista.	`[[${#lists.last(nombres)}]]`	`Marta`

`#lists.get(lista, índice)`	Devuelve el elemento en la posición indicada (0-based).	`[[${#lists.get(nombres, 1)}]]`	`Luis`
`#lists.concat(lista1, lista2)`	Une dos listas en una sola.	`[[${#lists.concat(List.of('A','B'), List.of('C'))}]]`	`[A, B, C]`
`#lists.sublist(lista, inicio, fin)`	Devuelve una sublista entre los índices dados (excluye `fin`).	`[[${#lists.sublist(nombres, 0, 2)}]]`	`[Ana, Luis]`
`#lists.union(lista1, lista2)`	Une dos listas **sin eliminar duplicados**.	`[[${#lists.union(List.of(1,2), List.of(2,3))}]]`	`[1, 2, 2, 3]`
`#lists.intersect(lista1, lista2)`	Devuelve los elementos **comunes** a ambas listas.	`[[${#lists.intersect(List.of(1,2,3), List.of(2,3,4))}]]`	`[2, 3]`
`#lists.diff(lista1, lista2)`	Devuelve los elementos de `lista1` que **no están** en `lista2`.	`[[${#lists.diff(List.of(1,2,3), List.of(2))}]]`	`[1, 3]`
`#lists.sum(lista)`	Suma los valores numéricos de la lista.	`[[${#lists.sum(List.of(2,4,6))}]]`	`12`
`#lists.avg(lista)`	Calcula el promedio de la lista.	`[[${#lists.avg(List.of(2,4,6))}]]`	`4.0`
`#lists.max(lista)`	Devuelve el valor máximo de la lista.	`[[${#lists.max(List.of(10,5,8))}]]`	`10`
`#lists.min(lista)`	Devuelve el valor mínimo de la lista.	`[[${#lists.min(List.of(10,5,8))}]]`	`5`
`#lists.shuffle(lista)`	Devuelve una nueva lista con los elementos en orden aleatorio.	`[[${#lists.shuffle(nombres)}]]`	`[Luis, Ana, Marta]` *(orden variable)*

Objeto #maps

Por categorías:

Categoría	Métodos más útiles
Información básica	`size()`, `isEmpty()`
Comprobaciones	`containsKey()`, `containsValue()`
Acceso	`get()`, `keys()`, `values()`, `entries()`
Combinación	`merge(map1, map2)`
Recorridos	`entries()` o `entrySet()` para usar en bucles `th:each`

Detalladas:

Método / Propiedad	Descripción	Ejemplo de uso	Resultado
`#maps.size(mapa)`	Devuelve el número de pares clave–valor.	`[[${#maps.size(datos)}]]`	`3`
`#maps.isEmpty(mapa)`	Devuelve `true` si el mapa está vacío.	`[[${#maps.isEmpty(datos)}]]`	`false`
`#maps.containsKey(mapa, clave)`	Comprueba si el mapa contiene una clave específica.	`[[${#maps.containsKey(datos, 'nombre')}]]`	`true`
`#maps.containsValue(mapa, valor)`	Comprueba si el mapa contiene un valor.	`[[${#maps.containsValue(datos, 'Carlos')}]]`	`true`
`#maps.get(mapa, clave)`	Devuelve el valor asociado a una clave.	`[[${#maps.get(datos, 'edad')}]]`	`25`
`#maps.keys(mapa)`	Devuelve una lista con todas las claves.	`[[${#maps.keys(datos)}]]`	`[nombre, edad, ciudad]`
`#maps.values(mapa)`	Devuelve una lista con todos los valores.	`[[${#maps.values(datos)}]]`	`[Carlos, 25, Madrid]`
`#maps.entries(mapa)`	Devuelve una lista de pares clave–valor.	`[[${#maps.entries(datos)}]]`	`[nombre=Carlos, edad=25, ciudad=Madrid]`

`#maps.merge(map1, map2)`	Combina dos mapas (las claves duplicadas se sobrescriben con las del segundo).	`[[${#maps.merge(mapa1, mapa2)}]]`	`{clave1=valor1, clave2=valor2}`
`#maps.keySet(mapa)`	Sinónimo de `keys()` (colección de claves).	`[[${#maps.keySet(datos)}]]`	`[nombre, edad, ciudad]`
`#maps.valueSet(mapa)`	Sinónimo de `values()` (colección de valores).	`[[${#maps.valueSet(datos)}]]`	`[Carlos, 25, Madrid]`
`#maps.entrySet(mapa)`	Sinónimo de `entries()` (lista de pares clave–valor).	`[[${#maps.entrySet(datos)}]]`	`[nombre=Carlos, edad=25]`

Objeto #bools

Por categoria:

Categoría	Métodos más útiles
Verificación lógica	`isTrue()`, `isFalse()`
Evaluación flexible (“truthy/falsy”)	`isTruthy()`, `isFalsy()`
Operaciones lógicas	`not()`, `and()`, `or()`, `xor()`, `nand()`, `nor()`
Comparaciones	`equals()`, `neq()`

Detalladas:

Método / Propiedad	Descripción	Ejemplo de uso	Resultado esperado
`#bools.isTrue(valor)`	Devuelve `true` si el valor es exactamente `true`.	`[[${#bools.isTrue(true)}]]`	`true`
`#bools.isFalse(valor)`	Devuelve `true` si el valor es exactamente `false`.	`[[${#bools.isFalse(false)}]]`	`true`
`#bools.isTruthy(valor)`	Evalúa si el valor es “verdadero” (true, no nulo, no vacío).	`[[${#bools.isTruthy('texto')}]]`	`true`

`#bools.isFalsy(valor)`	Evalúa si el valor es “falso” (false, nulo, vacío, 0, lista vacía, etc.).	`[[${#bools.isFalsy('')}]]`	`true`
`#bools.not(valor)`	Devuelve la negación lógica (`!valor`).	`[[${#bools.not(true)}]]`	`false`
`#bools.and(v1, v2)`	Devuelve `true` si ambos valores son verdaderos.	`[[${#bools.and(true, false)}]]`	`false`
`#bools.or(v1, v2)`	Devuelve `true` si al menos uno de los valores es verdadero.	`[[${#bools.or(true, false)}]]`	`true`
`#bools.xor(v1, v2)`	Devuelve `true` si **solo uno** de los valores es verdadero.	`[[${#bools.xor(true, false)}]]`	`true`
`#bools.nand(v1, v2)`	Negación de `and`: devuelve `false` solo si ambos son verdaderos.	`[[${#bools.nand(true, true)}]]`	`false`
`#bools.nor(v1, v2)`	Negación de `or`: devuelve `true` solo si ambos son falsos.	`[[${#bools.nor(false, false)}]]`	`true`
`#bools.equals(v1, v2)`	Comprueba si dos valores booleanos son iguales.	`[[${#bools.equals(true, false)}]]`	`false`
`#bools.neq(v1, v2)`	Comprueba si dos valores booleanos son distintos.	`[[${#bools.neq(true, false)}]]`	`true`

Otros objetos utilitarios

#arrays

Categoría	Métodos más útiles
Información básica	`length()`, `isEmpty()`, `contains()`, `containsAll()`
Acceso	`get()`, `first()`, `last()`
Manipulación	`sort()`, `reverse()`, `concat()`, `subarray()`
Conversión y visualización	`join()`, `asList()`, `toString()`

#aggregates

Categoría	Métodos más útiles
Cálculos numéricos	`sum()`, `avg()`, `min()`, `max()`, `median()`, `product()`
Análisis estadístico	`mode()` (moda), `count()` (conteo)
Operaciones de conjuntos	`distinct()`, `joinDistinct()`
Generación de secuencias	`range(from, to)`

#calendar (mas completo que #date)

Categoría	Métodos más útiles
Creación de calendarios	`createNow()`, `createToday()`, `create()`
Información de fecha y hora	`year()`, `month()`, `monthName()`, `day()`, `dayOfWeekName()`, `time()`
Operaciones temporales	`addDays()`, `addMonths()`, `addYears()`
Comparaciones	`isBefore()`, `isAfter()`, `isSameDay()`
Diferencias entre fechas	`diffDays()`, `diffMonths()`, `diffYears()`
Formato	`format()`, `formatISO()`

#temporals (para trabajar con LocalDate, LocalTime y LocalDateTime)

Categoría	Métodos más útiles
Creación de fechas y tiempos	`createNow()`, `createToday()`, `createDate()`, `createDateTime()`
Componentes de fecha y hora	`year()`, `month()`, `monthName()`, `day()`, `dayOfWeekName()`, `hour()`, `minute()`
Operaciones con fechas	`addDays()`, `addMonths()`, `addYears()`
Comparaciones	`isBefore()`, `isAfter()`, `isSameDay()`
Diferencias	`diffDays()`, `diffMonths()`, `diffYears()`
Formatos de salida	`format()`, `formatISO()`